CSED-321 Assignment 5 (100 points)

Due at 11:59pm, April 19, 2026 (Sunday)

In this assignment, you will implement the type system of TML (Typed ML) which is essentially the simply typed λ-calculus without the type void.

The abstract syntax for TML is shown below. Compared with the abstract syntax for the simply typed λ-calculus discussed in class, it includes another base type int for integers, but omits the type void which is useless in practice. $\widehat{n}$ denotes an integer n, and plus and minus are arithmetic operators on integers; eq tests two integers for equality.

$$
\begin{array}{rrcl}
\textsf{type} & A & ::= & \textsf{unit} \mid \textsf{bool} \mid \textsf{int} \mid A \to A \mid A \times A \mid A + A \\
\textsf{expression} & e & ::= & x \mid \lambda x{:}A.\, e \mid e\ e \mid \textsf{fix}\ x{:}A.\, e \mid \\
 & & & (e, e) \mid \textsf{fst}\ e \mid \textsf{snd}\ e \mid \\
 & & & \textsf{inl}_A\ e \mid \textsf{inr}_A\ e \mid \textsf{case}\ e\ \textsf{of}\ \textsf{inl}\ x.\, e \mid \textsf{inr}\ x.\, e \mid \\
 & & & () \mid \textsf{true} \mid \textsf{false} \mid \textsf{if}\ e\ \textsf{then}\ e\ \textsf{else}\ e \mid \widehat{n} \mid \textsf{plus} \mid \textsf{minus} \mid \textsf{eq} \\
\textsf{typing context} & \Gamma & ::= & \cdot \mid \Gamma, x : A
\end{array}
$$

The concrete syntax for TML is as follows. All entries in the right side of the definition below are arranged in the same order that their counterparts in the abstract syntax appear in the definition above:

$$
\begin{array}{rrcl}
\text{type} & A & ::= & \texttt{unit} \mid \texttt{bool} \mid \texttt{int} \mid A\ \texttt{->}\ A \mid A\ \texttt{*}\ A \mid A\ \texttt{+}\ A \mid \texttt{(}A\texttt{)} \\
\text{expression} & e & ::= & x \mid \texttt{fn}\ x{:}A\ \texttt{=>}\ e \mid e\ e \mid \texttt{fix}\ x{:}A\ \texttt{=>}\ e \mid \\
 & & & \texttt{(}e\texttt{,}\ e\texttt{)} \mid \texttt{fst}\ e \mid \texttt{snd}\ e \mid \\
 & & & \texttt{inl}\ \texttt{(}A\texttt{)}\ e \mid \texttt{inr}\ \texttt{(}A\texttt{)}\ e \mid \texttt{case}\ e\ \texttt{of}\ \texttt{inl}\ x\ \texttt{=>}\ e \mid \texttt{inr}\ x\ \texttt{=>}\ e \mid \\
 & & & \texttt{()} \mid \texttt{true} \mid \texttt{false} \mid \texttt{if}\ e\ \texttt{then}\ e\ \texttt{else}\ e \mid \widehat{n} \mid \texttt{+} \mid \texttt{-} \mid \texttt{=} \mid \\
 & & & \texttt{let}\ x{:}A\ \texttt{=}\ e\ \texttt{in}\ e' \mid \texttt{let rec}\ x{:}A\ \texttt{=}\ e\ \texttt{in}\ e' \mid \texttt{(}e\texttt{)} \\
\text{integer} & \widehat{n} & ::= & \texttt{0} \mid \texttt{1} \mid \texttt{2} \mid \cdots
\end{array}
$$

We add two forms of syntactic sugar:

$$
\begin{array}{rcl}
\texttt{let}\ x{:}A\ \texttt{=}\ e\ \texttt{in}\ e' & \text{for} & \texttt{(fn}\ x{:}A\ \texttt{=>}\ e'\texttt{)}\ e \\
\texttt{let rec}\ x{:}A\ \texttt{=}\ e\ \texttt{in}\ e' & \text{for} & \texttt{(fn}\ x{:}A\ \texttt{=>}\ e'\texttt{)}\ \texttt{(fix}\ x{:}A.\ e\texttt{)}
\end{array}
$$

$\texttt{(}A\texttt{)}$ is the same as A, and is used to alter the default right associativity of `->`. For example, A_1 `->` A_2 `->` A_3 is equal to A_1 `->` $\texttt{(}A_2$ `->` $A_3\texttt{)}$, which is different from $\texttt{(}A_1$ `->` $A_2\texttt{)}$ `->` A_3. Similarly, $\texttt{(}e\texttt{)}$ is the same as e, and is used to alter the default left associativity of applications. For example, $e_1\ e_2\ e_3$ is equal to $\texttt{(}e_1\ e_2\texttt{)}\ e_3$, which is different from $e_1\ \texttt{(}e_2\ e_3\texttt{)}$.

The goal of this assignment is to implement the type system of TML shown in Figure 1.

1 Programming Instructions

Download `hw5.zip` from PLMS, and unzip it on your working directory. It will create a bunch of files on the working directory.

First, see `tml.ml` which declares a structure `Tml`:

```
type var = string
type tp =
  ...
type exp =
  ...
```

$$\frac{x : A \in \Gamma}{\Gamma \vdash x : A}\ \mathsf{Var} \qquad \frac{\Gamma, x : A \vdash e : B}{\Gamma \vdash \lambda x{:}A.\, e : A \to B}\ \to\!\mathsf{I} \qquad \frac{\Gamma \vdash e : A \to B \quad \Gamma \vdash e' : A}{\Gamma \vdash e\ e' : B}\ \to\!\mathsf{E} \qquad \frac{\Gamma, x : A \vdash e : A}{\Gamma \vdash \mathsf{fix}\ x{:}A.\, e : A}\ \mathsf{Fix}$$

$$\frac{\Gamma \vdash e_1 : A_1 \quad \Gamma \vdash e_2 : A_2}{\Gamma \vdash (e_1, e_2) : A_1 \times A_2}\ \times\!\mathsf{I} \qquad \frac{\Gamma \vdash e : A_1 \times A_2}{\Gamma \vdash \mathsf{fst}\ e : A_1}\ \times\!\mathsf{E}_1 \qquad \frac{\Gamma \vdash e : A_1 \times A_2}{\Gamma \vdash \mathsf{snd}\ e : A_2}\ \times\!\mathsf{E}_2$$

$$\frac{\Gamma \vdash e : A_1}{\Gamma \vdash \mathsf{inl}_{A_2}\ e : A_1 + A_2}\ +\!\mathsf{I}_\mathsf{L} \qquad \frac{\Gamma \vdash e : A_2}{\Gamma \vdash \mathsf{inr}_{A_1}\ e : A_1 + A_2}\ +\!\mathsf{I}_\mathsf{R} \qquad \frac{\Gamma \vdash e : A_1 + A_2 \quad \Gamma, x_1 : A_1 \vdash e_1 : C \quad \Gamma, x_2 : A_2 \vdash e_2 : C}{\Gamma \vdash \mathsf{case}\ e\ \mathsf{of}\ \mathsf{inl}\ x_1.\, e_1 \mid \mathsf{inr}\ x_2.\, e_2 : C}\ +\!\mathsf{E}$$

$$\frac{}{\Gamma \vdash () : \mathsf{unit}}\ \mathsf{Unit} \qquad \frac{}{\Gamma \vdash \mathsf{true} : \mathsf{bool}}\ \mathsf{True} \qquad \frac{}{\Gamma \vdash \mathsf{false} : \mathsf{bool}}\ \mathsf{False} \qquad \frac{\Gamma \vdash e : \mathsf{bool} \quad \Gamma \vdash e_1 : A \quad \Gamma \vdash e_2 : A}{\Gamma \vdash \mathsf{if}\ e\ \mathsf{then}\ e_1\ \mathsf{else}\ e_2 : A}\ \mathsf{If}$$

$$\frac{}{\Gamma \vdash \widehat{n} : \mathsf{int}}\ \mathsf{Int} \qquad \frac{}{\Gamma \vdash \mathsf{plus} : \mathsf{int} \times \mathsf{int} \to \mathsf{int}}\ \mathsf{Plus} \qquad \frac{}{\Gamma \vdash \mathsf{minus} : \mathsf{int} \times \mathsf{int} \to \mathsf{int}}\ \mathsf{Minus} \qquad \frac{}{\Gamma \vdash \mathsf{eq} : \mathsf{int} \times \mathsf{int} \to \mathsf{bool}}\ \mathsf{Eq}$$

Figure 1: Type system of TML

The datatypes `tp` and `exp` correspond to the syntactic categories type and expression, respectively. Read the comments in the file and make sure that you understand what each data constructor is for.

Next, see `typing.mli` and `typing.ml`. The goal is to implement the function `typing` in the structure `Typing`:

```
val typing : context -> Tml.exp -> Tml.tp
```

That is, `typing` takes a typing context Γ of type `Typing.context` and an expression e of type `Tml.exp`, and returns a type A such that $\Gamma \vdash e : A$; it raises an exception `TypeError` if e does not typecheck, *i.e.*, there is no A such that $\Gamma \vdash e : A$.

Note that you will decide the definition of type `Typing.context` yourself. In the stub file, `Typing.context` is defined as `unit`, but you should replace it by an appropriate type for typing contexts. After deciding the definition of `Typing.context`, you have to implement the function `createEmptyContext` which returns an empty typing context:

```
val createEmptyContext : unit -> context
```

Then try to implement the rule Var to make sure that you can retrieve an individual element (which is a type binding $x : A$) from a typing context.

You have to think about how to represent $\Gamma, x : A$ which is required by the rules $\to\!\mathsf{I}$, $+\!\mathsf{E}$, and Fix. If you are tempted to use `(Tml.var * Tml.tp) list` for `Typing.context`, think again. `(Tml.var * Tml.tp) list` is definitely a good candidate for the definition of `Typing.context` (and indeed a reasonably good solution), but try to come up with a better and more elegant definition.

After implementing the function `typing` in `typing.ml`, run `make` to compile the source files.

```
$ make
ocamlc -thread  -c tml.ml -o tml.cmo
ocamlyacc parser.mly
26 shift/reduce conflicts.
ocamlc -c parser.mli
ocamlc -c parser.ml
ocamllex lexer.mll
67 states, 4625 transitions, table size 18902 bytes
ocamlc -c lexer.ml
ocamlc -thread  -c typing.mli -o typing.cmi
ocamlc -thread  -c typing.ml -o typing.cmo
ocamlc -thread  -c inout.ml -o inout.cmo
ocamlc -thread  -c eval.ml -o eval.cmo
ocamlc -thread  -c loop.ml -o loop.cmo
ocamlc -thread  -c hw5.ml -o hw5.cmo
```

```
ocamlc -o lib.cma -a tml.cmo parser.cmo lexer.cmo typing.cmo inout.cmo eval.cmo loop.cmo
ocamlc -o hw5 lib.cma hw5.cmo
```

There are two ways to test your code in `typing.ml`. First, you can run `hw5`. At the TML prompt, enter a TML expression followed by the semicolon symbol `;`. (The syntax of TML was given on page 1.) The type of the expression is displayed.

```
$ ./hw5
Tml> fn : int => x ;
Syntax error
Tml> fn x : int => x ;
(fn x : int => x) : int -> int
Tml> x;
x has no type.
```

Alternatively, you can use those functions in `loop.ml` in the interactive mode of OCaml. (You don't actually need to read `loop.ml`.) At the OCaml prompt, type `#load "lib.cma";;` to load the library for this assignment. Then open the structure `Loop`:

```
$ ocaml
OCaml version 4.14.2
Enter #help;; for help.

# #load "lib.cma";;
# open Loop;;
#
```

You type `loop (step show);;` at the OCaml prompt, and then enter a TML expression followed by the semicolon symbol `;`.

```
# loop (step show);;
Tml> fn x : int => x ;
(fn x : int => x) : int -> int
Tml> x;
x has no type.
```

2 Submission Instructions

- Make sure that you can compile `typing.ml` by running `make`. **You will receive 0 points if your code does not compile.**
- When you have the file `typing.ml` ready for submission, **upload only this file to PLMS using the filename** `typing.ml`.